

Universidade Estadual de Campinas
Faculdade de Tecnologia – FT

FT – UNICAMP

Relatório Técnico
SI405 – Padrões Estruturais: Adapter e Decorator

Tema: Sistema de Cobrança Corporativa

Produto: refatoração arquitetural em Java

Discentes:

Daniel Aniceto Rosell – RA 283988

Davie Schimidt Fonseca – RA 259908

Hugo Strassa – RA 246710

Gabriel Sorensen M. Traina – RA 283997

Kaue Samuel Oliveira da Silva – RA 178449

Kauã Henrique da Silva Andrade – RA 246165

Limeira – SP
2026

1. Introdução

Este relatório apresenta a solução desenvolvida para a atividade prática de SI405 sobre padrões estruturais, com foco nos padrões *Adapter* e *Decorator*. O sistema-base representa uma aplicação corporativa de cobrança de pedidos, originalmente capaz de processar pagamentos por boleto, Pix e cartão de crédito, além de aplicar ajustes de valor como desconto, juros, taxa internacional e seguro.

A implementação inicial estava funcional, porém apresentava problemas arquiteturais importantes. A integração com gateways de pagamento era feita diretamente no serviço principal, inclusive com código específico para bibliotecas externas. Além disso, os ajustes de valor eram controlados por parâmetros booleanos, o que dificultava a evolução da aplicação quando novas taxas precisassem ser adicionadas.

Repositório GitHub da solução: <https://github.com/SorensenG/si405-padroes-estruturais-grupo>

O desenvolvimento foi organizado entre 18/06/2026 às 14:00 e 25/06/2026. A solução preservou o projeto original, manteve os testes existentes funcionando e adicionou suporte à forma de pagamento Carteira Digital e aos novos ajustes exigidos no enunciado.

2. Análise dos principais problemas arquiteturais

O primeiro problema identificado foi o acoplamento do serviço de cobrança às implementações concretas dos gateways. A classe *CobrancaService* conhecia diretamente o gateway interno, o SDK externo *PaySecureGateway*, o formato dos dados exigidos por esse SDK e o tratamento de exceções específico da integração. Dessa forma, uma classe de regra de aplicação passou a depender de detalhes de infraestrutura.

Outro problema estava na duplicação da lógica de seleção e conversão de gateways. Os métodos *cobrar* e *cobrarEmLote* repetiam a mesma estrutura condicional para boleto, Pix e cartão de crédito. Assim, qualquer nova forma de pagamento exigiria alterações em mais de um ponto, aumentando o risco de inconsistência.

No cálculo de valor final, a aplicação usava parâmetros booleanos para indicar quais ajustes deveriam ser aplicados. Essa solução funcionava para poucos casos, mas não era extensível. Cada novo ajuste exigiria alterar a assinatura de métodos, adicionar novos condicionais e espalhar a regra pelo serviço.

Problema central: o sistema funcionava, mas misturava seleção de gateway, adaptação de SDK externo, cálculo de valor e composição de ajustes dentro do mesmo fluxo de serviço.

3. Justificativa das decisões arquiteturais adotadas

A solução adotada foi uma refatoração localizada, mantendo o código-base fornecido e evitando substituir o sistema por uma implementação nova. A prioridade foi preservar o comportamento existente, reduzir acoplamento e criar pontos claros de extensão.

Para os meios de pagamento, foi utilizado o padrão *Adapter*. O serviço de cobrança passou a depender de uma interface comum de gateway, enquanto cada integração concreta ficou encapsulada em uma classe adaptadora. Essa decisão atende diretamente à restrição de não modificar classes do pacote externo, como *PaySecureGateway* e *WalletPaySDK*.

Para os ajustes de valor, foi utilizado o padrão *Decorator*. Cada ajuste passou a ser representado por um objeto independente, capaz de envolver o valor calculado anteriormente e acrescentar sua regra. Com isso, a aplicação pode combinar ajustes em qualquer quantidade e ordem sem aumentar a assinatura principal de cobrança.

4. Aplicação do padrão Adapter

O padrão *Adapter* foi aplicado por meio da interface *GatewayCobranca*. Essa interface define uma operação comum de cobrança, recebendo o pedido, o valor final e a forma de pagamento. O *CobrancaService* deixa de conhecer diretamente os SDKs e passa a depender apenas dessa abstração.

Foram criados três adaptadores principais. O *GatewayPagamentoInternoAdapter* adapta o gateway interno usado por boleto e Pix. O *PaySecureGatewayAdapter* adapta o SDK externo de cartão de crédito. O *WalletPayGatewayAdapter* integra o novo SDK de Carteira Digital solicitado no enunciado.

No caso do *PaySecureGateway*, o adapter centraliza a criação do mapa de dados esperado pelo SDK, incluindo identificador do pedido, nome do cliente, valor e moeda. Também converte o código de status externo para os status internos *APROVADA* e *RECUSADA*, além de tratar a exceção de indisponibilidade do gateway.

No caso do *WalletPaySDK*, o adapter converte o valor de reais para centavos, cria a requisição exigida pelo SDK e traduz os status externos. O status *CONFIRMED* é tratado como aprovado; os status *DECLINED* e *FAILED* são tratados como recusados.

Componente	Responsabilidade
<i>GatewayCobranca</i>	Interface comum usada pelo serviço de cobrança.
<i>GatewayPagamento</i>	Adapta boleto e Pix para a interface comum.
<i>InternoAdapter</i>	
<i>PaySecureGatewayAdapter</i>	Isola a integração com o SDK externo de cartão.
<i>WalletPayGatewayAdapter</i>	Integra a nova forma de pagamento Carteira Digital.
<i>GatewayCobrancaResolver</i>	Centraliza a escolha do adapter adequado para cada forma de pagamento.

Tabela 1: Organização da solução baseada em Adapter.

5. Aplicação do padrão Decorator

O padrão *Decorator* foi aplicado no cálculo do valor final da cobrança. O componente base é representado por *ValorCobranca*, e a classe *ValorBaseCobranca* encapsula o valor inicial do pedido. A classe abstrata *AjusteValorDecorator* serve como base para os decorators concretos.

Os ajustes já existentes foram preservados nas classes *DescontoFidelidadeDecorator*, *JurosParcelamentoDecorator*, *TaxaOperacaoInternacionalDecorator* e *SeguroTransacaoDecorator*. As regras originais foram mantidas: desconto de 5%, juros de 2,99%, taxa internacional de 5% e seguro fixo de R\$ 4,90.

Também foram adicionados os novos ajustes solicitados: *TaxaAntecipacaoRecebiveisDecorator*, com taxa de 1,5%, e *TaxaEmissaoNotaFiscalDecorator*, com valor fixo de R\$ 2,50. O serviço passou a aceitar uma lista de ajustes, permitindo montar a cadeia de decorators dinamicamente.

Compatibilidade: os métodos antigos com parâmetros booleanos foram mantidos como camada de compatibilidade, mas internamente delegam para a nova composição orientada a objetos.

Decorator	Regra aplicada
Desconto de fidelidade	Reduz 5% do valor.
Juros de parcelamento	Acrescenta 2,99% ao valor.
Taxa internacional	Acrescenta 5% ao valor.
Seguro de transação	Acrescenta R\$ 4,90.
Antecipação de recebíveis	Acrescenta 1,5% ao valor.
Emissão de nota fiscal	Acrescenta R\$ 2,50.

Tabela 2: Ajustes representados como decorators.

6. Impactos da refatoração

A refatoração reduziu o acoplamento entre o serviço de cobrança e os gateways concretos. Para adicionar uma nova forma de pagamento, basta criar um novo adapter e registrá-lo no resolvedor. O serviço principal permanece estável e não precisa conhecer os detalhes de cada SDK.

A duplicação entre cobrança individual e cobrança em lote também foi reduzida. Os dois fluxos reutilizam a mesma lógica de cálculo e a mesma resolução de gateway, evitando que mudanças futuras precisem ser repetidas manualmente em mais de um método.

Com o uso de *Decorator*, os ajustes de valor ficaram independentes, combináveis e testáveis isoladamente. A criação de novas taxas não exige novos parâmetros booleanos nem condicionais no cálculo central. Essa organização favorece manutenção e extensibilidade, além de demonstrar o princípio estudado em aula: preferir composição sobre estruturas rígidas de decisão.

7. Testes executados

Os testes existentes foram mantidos e novos cenários foram adicionados para cobrir os requisitos do enunciado. Foram criados testes para o adapter do *PaySecureGateway*, para o adapter do *WalletPaySDK*, para os novos decorators e para combinações de múltiplos decorators em ordens diferentes.

Resultado da validação: a execução de `mvn test` finalizou com 34 testes executados, 0 falhas e 0 erros. Também foi executado `mvn clean package -q` com sucesso.

Categoria de teste	Cenários validados
Testes existentes	Pedido, gateway interno, PaySecure, WalletPay e serviço de cobrança.
Adapters	Aprovação e recusa em cartão e Carteira Digital.
Decorators novos	Antecipação de recebíveis e emissão de nota fiscal.
Composição	Combinação de múltiplos decorators em ordens distintas.

Tabela 3: Resumo dos testes automatizados.

8. Divisão de responsabilidades

Parte	Responsáveis
Estrutura base de adapters e gateway interno	Daniel, Gabriel
Adapter do PaySecureGateway	Davie, Gabriel
Integração do WalletPaySDK e Carteira Digital	Hugo, Gabriel
Refatoração do CobrancaService e resolvidor de gateways	Daniel, Davie, Hugo, Gabriel
Decorators dos ajustes existentes	Kaue, Gabriel
Decorators novos, testes e revisão final	Kauã, Kaue, Gabriel e demais integrantes

Tabela 4: Organização do trabalho do grupo.